Design Patterns în Java — Exerciții

Adrian · Strategy · Observer · Factory Method · Decorator · Builder · Singleton

Cum lucrezi

- Șase exerciții, câte unul pe pattern: `src/app/ex1` … `src/app/ex6`. Fiecare conține un cod care **funcționează și afișează ce trebuie**, dar e scris prost. Rolul tău e să-l refactorizezi, fără să schimbi ce afișează.
- Fiecare exercițiu are doi pași: **Pasul 1**, refactorizarea (ieșirea scenariului de azi rămâne rând cu rând identică), și **Pasul 2**, o cerință nouă pe care codul refactorizat o primește ușor.
- În `src/exemple/` ai câte un program mic și curat pentru fiecare pattern, pe alt domeniu decât exercițiul: `strategy`, `observer`, `factory`, `decorator`, `builder`, `singleton`. Rulează-l și citește-l înainte de exercițiu. Nu-l copia: domeniul e altul.
- Teoria e în `TEORIE.pdf`. `Main` (cheamă `Testare1` … `Testare6`) nu-l modifici.
- Rulezi din IntelliJ (click dreapta pe `Main` → Run) sau din terminal, în rădăcina proiectului:

```
javac -d out $(find src -name '*.java')
java -cp out app.Main
java -cp out exemple.strategy.Main
```

Reguli pentru toate: fără `instanceof` și `getClass()`, mesajele de eroare în engleză. Ordinea: `ex1` → `ex6`.

Exercițiul 1 · Livrare

pachetul `app.ex1`

Codul de azi:

`Comanda` calculează costul livrării unei comenzi: standard, express sau gratuită, după `TipLivrare`. Rulează `Testare1` (prin `Main`) și citește `Comanda`.

Întrebare: câte locuri din `Comanda` trebuie să editezi dacă mâine apare o a patra metodă de livrare?

Pattern-ul:

Strategy. Exemplul curat: `exemple/strategy/` (metode de plată). Teorie: lecția 1 din `TEORIE.pdf`.

Pasul 1 · refactorizezi:

- Un contract `LivrareStrategie` cu `String getNume()` și `double calculeazaCost(double greutateKg, double distantaKm)`.
- Clasele `LivrareStandard`, `LivrareExpress`, `LivrareGratuita` îl implementează; fiecare ține doar formula ei.
- `Comanda` primește strategia prin constructor, în locul lui `TipLivrare`. Metodele `numeLivrare()` și `cost()` rămân, cu aceleași nume, și deleagă strategiei. Fără `switch` / `if` pe tip.
- Adaptezi `Testare1` la noul constructor; scenariul rămâne același (aceeași comandă de 10 kg, 100 km, cu cele trei metode, apoi greutatea 0).

Ieșirea (identică cu cea de azi):

```
Standard: 20.0
Express: 47.0
Gratuita: 0.0
Weight must be positive
```

Pasul 2 · cerința nouă:

Adaugi livrarea internațională: nume `Internationala`, cost `30 + 2.0 * greutate + 0.5 * distanta`. La finalul lui `Testare1`, aceeași comandă (Ana, 10 kg, 100 km) cu ea. Rândul nou:

```
Internationala: 100.0
```

Gata când:

- Primele 4 rânduri sunt identice cu cele de azi (diff gol).
- Livrarea nouă a însemnat o clasă nouă și o linie în `Testare1`; nicio altă clasă nu a fost modificată.

Exercițiul 2 · Starea comenzii

pachetul `app.ex2`

Codul de azi:

`Comanda` își schimbă starea (`Plasata → Expediata → Livrata`), iar la fiecare schimbare se întâmplă mai multe lucruri: email, jurnal, depozit. Rulează `Testare2` și citește `Comanda`.

Întrebare: ce clase concrete cunoaște `Comanda`, și ce editezi în ea când vrei să anunți și prin SMS?

Pattern-ul:

Observer. Exemplul curat: `exemple/observer/` (canal video). Teorie: lecția 2.

Pasul 1 · refactorizezi:

- Un contract `Observator` cu `void actualizeaza(String stareNoua)`; cele trei clase (email, jurnal, depozit) îl implementează și își păstrează afișările.
- `Comanda` nu mai creează și nu mai cunoaște nicio clasă concretă: primește `aboneaza(Observator)` și `dezaboneaza(Observator)` și îi anunță pe toți la schimbare. Validarea (`State cannot be empty`) și regula „aceeași stare = nu anunți” rămân.
- În `Testare2` abonezi cele trei, în ordinea email, jurnal, depozit; apoi aceeași secvență de stări ca azi.

Ieșirea (identică cu cea de azi):

```
[EMAIL] Comanda a trecut in starea: Expediata
[LOG] Stare inregistrata: Expediata
[DEPOZIT] Pregatesc pentru: Expediata
[EMAIL] Comanda a trecut in starea: Livrata
[LOG] Stare inregistrata: Livrata
[DEPOZIT] Pregatesc pentru: Livrata
Stare finala: Livrata
State cannot be empty
```

Pasul 2 · cerința nouă:

Adaugi anunțul prin SMS: clasa `NotificatorSms`, care afișează `[SMS] Comanda a trecut in starea: X`. La finalul lui `Testare2` o abonezi și schimbi starea în `"Returnata"`. Rândurile noi:

```
[EMAIL] Comanda a trecut in starea: Returnata
[LOG] Stare inregistrata: Returnata
[DEPOZIT] Pregatesc pentru: Returnata
[SMS] Comanda a trecut in starea: Returnata
```

Gata când:

- Primele 8 rânduri sunt identice cu cele de azi (diff gol).
- SMS-ul a însemnat o clasă nouă și două linii în `Testare2`; `Comanda` nu a fost atinsă.

Exercițiul 3 · Încărcarea utilizatorilor

pachetul `app.ex3`

Codul de azi:

`IncarcatorUtilizatori` transformă linii de forma `TIP,Prenume,Nume,Detaliu` în obiecte `Student` sau `Profesor`. Rulează `Testare3` și citește încărcătorul.

Întrebare: ce fișiere atingi dacă mâine apare tipul `ADMIN`? De ce nu ajută faptul că `Student` și `Profesor` își suprascriu `descriere()`?

Pattern-ul:

Factory Method. Exemplul curat: `exemple/factory/` (forme). Teorie: lecția 3.

Pasul 1 · refactorizezi:

- Un contract `FabricaUtilizator` cu `String getTip()` și `Utilizator creeaza(String[] campuri)`; `FabricaStudent` și `FabricaProfesor` îl implementează, fiecare cu un singur `new`.
- `IncarcatorUtilizatori` primește lista de fabrici prin constructor. Pentru fiecare linie caută fabrica al cărei `getTip()` e egal cu primul câmp; dacă niciuna, `IllegalArgumentException("Unknown user type: " + tip)`.
- În `IncarcatorUtilizatori` nu mai apare `new Student`, `new Profesor` și niciun `switch`. Singura comparație permisă e între două șiruri.
- Adaptezi `Testare3`: încărcătorul primește `List.of(new FabricaStudent(), new FabricaProfesor())`; liniile rămân aceleași.

Ieșirea (identică cu cea de azi):

```
[STUDENT] Ana Popescu – anul 2
[PROFESOR] Ion Ionescu – Matematica
[STUDENT] Radu Marin – anul 1
Unknown user type: SECRETAR
```

Pasul 2 · cerința nouă:

Adaugi tipul `ADMIN`: clasa `Admin` (extinde `Utilizator`, are un câmp `departament`, iar `descriere()` întoarce `[ADMIN] Prenume Nume – departament`) și `FabricaAdmin` (`getTip()` = `"ADMIN"`). La finalul lui `Testare3` creezi un al doilea încărcător, cu cele trei fabrici, și încarci linia `ADMIN,Mihai,Dumitru,IT`. Rândul nou:

```
[ADMIN] Mihai Dumitru – IT
```

Gata când:

- Primele 4 rânduri sunt identice cu cele de azi (diff gol).
- `ADMIN` a însemnat două clase noi; în `IncarcatorUtilizatori` nu ai modificat nicio linie.

Exercițiul 4 · Notificări

pachetul `app.ex4`

Codul de azi:

O aplicație trimite notificări prin email sau SMS, cu prefix, trunchiere la 30 de caractere sau jurnalizare. Clasele sunt în `ex4/models`. Rulează `Testare4` și numără clasele.

Întrebare: câte clase noi ți-ar trebui pentru un SMS cu prefix? Dar pentru un SMS cu prefix, jurnal și trunchiere?

Pattern-ul:

Decorator. Exemplul curat: `exemple/decorator/` (cafea cu topping-uri). Teorie: lecția 4.

Pasul 1 · refactorizezi:

- Păstrezi `Notificator`, `NotificatorEmail`, `NotificatorSms`. Cele patru clase combinatorii dispar.
- În locul lor, trei clase care semnează `Notificator` și țin înăuntru alt `Notificator`: `CuPrefix(String prefix, Notificator interior)`, `CuTrunchiere(int lungimeMaxima, Notificator interior)`, `CuJurnal(Notificator interior)`.
- `getNume()` nu întoarce un nume fix: compune numele din interior, cu `" + prefix"`, `" + trunchiere"`, `" + jurnal"`. Jurnalul afișează `[JURNAL] catre <nume interior>: <mesaj>` și apoi trimite mai departe mesajul neschimbat.
- Adaptezi `Testare4`: aceleași cinci scenarii, acum din lanțuri de obiecte (jurnalul în afară, trunchierea înăuntru).

Ieșirea (identică cu cea de azi):

```
Email + prefix
[EMAIL] [URGENT] Tema e corectata
Message exceeds SMS length limit
SMS + trunchiere
[SMS] Tema noua: Observer, de predat
SMS + jurnal
[JURNAL] catre SMS: Tema e corectata
[SMS] Tema e corectata
SMS + trunchiere + jurnal
[JURNAL] catre SMS + trunchiere: Tema noua: Observer, de predat vineri
[SMS] Tema noua: Observer, de predat
```

Pasul 2 · cerința nouă:

Adaugi decoratorul `CuMajuscule` (nume `" + majuscule"`, trimite mesajul cu majuscule). La finalul lui `Testare4`: un email cu prefix `[URGENT]`, împachetat în `CuMajuscule`; afișezi numele, apoi trimiți `"Tema e corectata"`. Rândurile noi:

```
Email + prefix + majuscule
[EMAIL] [URGENT] TEMA E CORECTATA
```

Gata când:

- Primele 11 rânduri sunt identice cu cele de azi (diff gol).
- Decoratorul nou a însemnat o clasă; nicio clasă existentă nu a fost modificată, iar combinația cu prefix a ieșit din obiecte, nu dintr-o clasă în plus.

Exercițiul 5 · Comandă cu opționale

pachetul `app.ex5`

Codul de azi:

`Comanda` are doi parametri obligatorii (client, adresă) și patru opționali (notă, cupon, express, cadou), toți într-un singur constructor. Rulează `Testare5`.

Întrebare: ce înseamnă `null, null, false, false` din primul apel, fără să deschizi clasa? Ce se întâmplă dacă inversezi nota cu cuponul?

Pattern-ul:

Builder. Exemplul curat: `exemple/builder/` (pizza). Teorie: lecția 5.

Pasul 1 · refactorizezi:

- `Comanda` primește o clasă internă statică `Builder` cu metodele `client(String)`, `adresa(String)`, `nota(String)`, `cupon(String)`, `express()`, `cadou()` și `build()`. Fiecare setter întoarce `this`.
- Constructorul lui `Comanda` devine `private`; câmpurile rămân `final`, fără setteri.
- Validările se mută în `build()`, în aceeași ordine și cu aceleași mesaje (`Client is required`, `Address is required`, `Gift wrap requires a note`).
- Rescrii `Testare5` cu builderul: aceleași trei comenzi și aceleași două cazuri invalide.

Ieșirea (identică cu cea de azi):

```
Comanda Ana -> Str. Florilor 5
Comanda Ion -> Bd. Unirii 10 | nota: Suna la interfon | cupon: TOAMNA10
Comanda Maria -> Str. Mare 1 | nota: La multi ani! | cupon: CADOU5 | express | cadou
Address is required
Gift wrap requires a note
```

Pasul 2 · cerința nouă:

Adaugi câmpul opțional `telefon` (`String`): setterul `telefon(String)` în builder, iar `descriere()` îl afișează la final, ca `| telefon: X`. La finalul lui `Testare5`: Dan, `Str. Lunga 2`, telefon `0712345678`. Rândul nou:

```
Comanda Dan -> Str. Lunga 2 | telefon: 0712345678
```

Gata când:

- Primele 5 rânduri sunt identice cu cele de azi (diff gol).
- Câmpul nou nu a cerut modificarea niciunui apel existent din `Testare5`, iar `new Comanda(...)` scris în afara clasei nu compilează.

Exercițiul 6 · Numerele documentelor

pachetul `app.ex6`

Codul de azi:

Comenzile și returnurile primesc numere din aceeași serie (`1001`, `1002` …), date de `GeneratorNumere`. Generatorul e creat o singură dată, în `Testare6`, și trimis mai departe prin constructori. Rulează `Testare6` și citește `Comanda`, `Retur`, `GeneratorNumere`.

Întrebare: ce se întâmplă cu numerele dacă cineva scrie, într-o clasă nouă, `new GeneratorNumere()`? Ce primește constructorul fiecărei clase doar ca să-l ducă mai departe?

Pattern-ul:

Singleton. Exemplul curat: `exemple/singleton/` (jurnal de aplicație). Teorie: lecția 6.

Pasul 1 · refactorizezi:

- `GeneratorNumere` devine singurul său creator: constructor `private` și o metodă `static GeneratorNumere getInstance()` care întoarce mereu aceeași instanță. Metoda `urmatorul()` rămâne.
- `Comanda` și `Retur` nu mai primesc generatorul prin constructor: îl iau singure, prin `getInstance()`. Constructorii lor pierd parametrul `GeneratorNumere`.
- În `Testare6` dispare variabila `generator`; scenariul rămâne același (Ana, Ion, returul pentru Ana, apoi Maria).

Ieșirea (identică cu cea de azi):

```
Comanda #1001 Ana
Comanda #1002 Ion
Retur #1003 pentru comanda #1001
Comanda #1004 Maria
```

Pasul 2 · cerința nouă:

Adaugi documentul `Factura`: se creează pentru o comandă (`new Factura(Comanda)`), ia un număr din aceeași serie și are `descriere()` = `Factura #N pentru comanda #M`. La finalul lui `Testare6` emiți o factură pentru Maria, apoi una pentru Ion. Rândurile noi:

```
Factura #1005 pentru comanda #1004
Factura #1006 pentru comanda #1002
```

Gata când:

- Primele 4 rânduri sunt identice cu cele de azi (diff gol).
- Factura a însemnat o clasă nouă și patru linii în `Testare6`; `GeneratorNumere`, `Comanda` și `Retur` nu au fost atinse, iar `new GeneratorNumere()` scris în afara clasei nu compilează.